COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	July 9, 2026	**DATE SUBMITTED:**	July 14, 2026
NAME:	Dela Cruz, Jared Isaiah P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1.1 **Basic C++ Programming**

1. PROCEDURE OUTPUT

ILO A: Create Code That Follows the Basic C++ Code Structure

For this activity, you have to demonstrate the use of a function prototype. The section on class declaration and method definition will be used for the function prototype and the function will be defined in the following method definition section after the main function.

A function prototype in C++ is a declaration of the name, parameters and return type of the function before its definition. Write a C++ code the satisfies the following:

- Create a function that will take two numbers and display the sum.
- Create a function that will return whether variable A is greater than variable B.
- Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

```cpp
#include <iostream>

void TwoNumSum(int fNum, int sNum);
bool isGreater(int A, int B);
bool logicalOperations (bool boolA, bool boolB);

int main() {
TwoNumSum(5, 10);

std::cout << std::endl;

if (isGreater(21, 25)) {
    std::cout << "Returned true." << std::endl;
} else {
    std::cout << "Returned false." << std::endl;
}

std::cout << std::endl;

if (logicalOperations(true, false)) {
    std::cout << "Logic operations success." << std::endl;
}
return 0;

}

void TwoNumSum(int fNum, int sNum) {
    std::cout << "The sum of the two numbers is: " << (fNum + sNum) << std::endl;
}

bool isGreater(int A, int B) {
    if (A > B) {
        std::cout << A << " is greater than " << B << std::endl;
        return true;
    } else {
        std::cout << A << " is less than or equal to " << B << std::endl;
        return false;
    }
}

bool logicalOperations (bool boolA, bool boolB) {
    std::cout << "A AND B: " << (boolA && boolB) << std::endl;
    std::cout << "A OR B: " << (boolA || boolB) << std::endl;
    std::cout << "A XOR B: " << (boolA ^ boolB) << std::endl;
    std::cout << "A Complement: " << (!boolA) << std::endl;
    std::cout << "B Complement: " << (!boolB) << std::endl;

    return true;
}
```

```
The sum of the two numbers is: 15

21 is less than or equal to 25
Returned false.

A AND B: 0
A OR B: 1
A XOR B: 1
A Complement: 0
B Complement: 1
Logic operations success.

--------------------------------
Process exited after 0.1856 seconds with return value 0
Press any key to continue . . .
```

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

In this section, the initial implementation for a class triangle will be implemented. The step-by-step procedure
is shown below:
Step 1. Include the necessary header files. For this one, we only need #include <iostream>
Step 2. Create the triangle class. Assign it with private variables: totalAngle, angleA, angleB, and angleC.

```
class Triangle{
private:
        double totalAngle, angleA, angleB, angleC;
```

Step 3. We then create public methods. The constructor must allow for creation of the object with 3 initial angles to be stored in our previously defined variables angleA, angleB and angleC. Another method has to be made if the user wants to change the initial values, this will also accept 3 arguments to change the values in angleA, angleB and angleC. Lastly, a function to validate whether the given values make our shape an actual triangle.

```
public:
      Triangle(double A, double B, double C);
      void setAngles(double A, double B, double C);
      const bool validateTriangle();
}
```

Step 4. Define the methods.

```
Triangle::Triangle(double A, double B, double C) {
angleA = A
angleB = B;
angleC = C;
totalAngle = A+B+C;
}

void Triangle::setAngles(double A, double B, double C) {
angleA = A;
angleB = B;
angleC = C;
totalAngle = A+B+C;
}

const bool Triangle::validateTriangle() {
return (totalAngle <= 180);
}
```

Step 5. Create the driver code.

```
int main(){
    //driver code
    Triangle set1(40, 30, 110);
    if(set1.validateTriangle()){
    std::cout << "The shape is a valid triangle.\n";
    } else {
    std::cout << "The shape is NOT a valid triangle.\n";
    }
    return 0;
}
```

Include the output of running this code in section 6. Note your observations and comments.

```cpp
#include <iostream>

class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC, sideA, sideB, sideC;
    public:
        Triangle(double A, double b, double C);
        void setAngles(double A, double B, double C);
        const bool validateTriangle();
};

int main(){
    Triangle set1(40, 30, 150);

    if(set1.validateTriangle()){
        std::cout<< "The shape is a valid triangle \n";
    }
    else {
        std::cout<< "The shape is NOT a valid triangle \n";
    }
}

Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

const bool Triangle::validateTriangle(){
    return(totalAngle <= 180);
}
```

```
The shape is NOT a valid triangle

--------------------------------
Process exited after 0.1843 seconds with return value 0
Press any key to continue . . .
```

Sections	Answer
Header File Declaration Section	`1 #include <iostream>`
Global Declaration Section	N/A
Class Declaration and Method Definition Section	`3 class Triangle{`

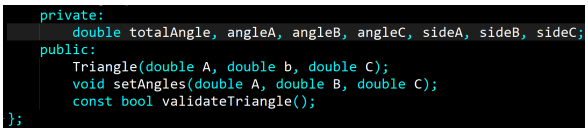
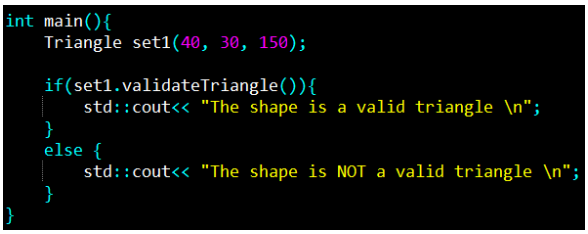
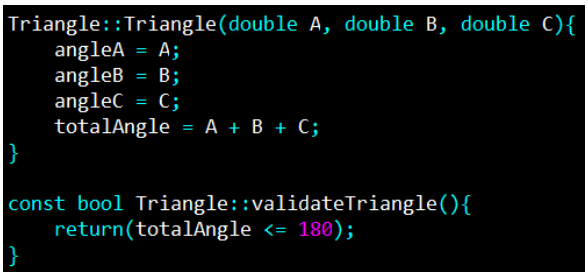
	
Main Function	
Method Definition	

Table 1-1. C++ Structure Code for Answer

OBSERVATION: For this specific line of code, it checks whether the sum of the provided angle values equate to a valid triangle. If the input we integrated is less than or equal to 180 degrees, then it will return true or is a valid triangle. Otherwise, it will return false or it counts as not a valid triangle.

Table 1-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language
The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.
1. Create a C++ program to swap the two numbers in different variables.

```cpp
#include <iostream>

int fNum = 0, sNum = 0, temp = 0;

int main() {

    std::cout << "Enter First No.: ";
    std::cin >> fNum;
    std::cout << "Enter Second No.: ";
    std::cin >> sNum;

    temp = fNum;
    fNum = sNum;
    sNum = temp;

    std::cout << "\nNumbers Given are Swapped.\n";
    std::cout << "First Number: " << fNum << std::endl;
    std::cout << "Second Number: " << sNum << std::endl;

return 0;
}
```

```
Enter First No.: 12
Enter Second No.: 24

Numbers Given are Swapped.
First Number: 24
Second Number: 12

--------------------------------
Process exited after 2.357 seconds with return value 0
Press any key to continue . . .
```

DIFFICULTIES/OBSERVATIONS: By utilizing temporary values, the given program demonstrates a swapping of the variables. The fNum works as a user-input prompt, meaning the user will type in the variables and will store those values. After that, the sNum will then overwrite the fNum input and assign the stored value to sNum, effectively swapping both values.

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```cpp
#include <iostream>

float KelvinToFahrenheit (float K);

int main() {
    float K;
    std::cout << "To convert into Fahrenheit, please enter Temperature in Kelvin: ";
    std::cin >> K;
    std::cout << "Temperature in Fahrenheit: " << kelvinToFahrenheit (K);
    return 0;
}

float KelvinToFahrenheit (float K){
    float F = ((K - 273.15) * (9.0/5.0) + 32.0);
    return F;
}
```

```
To convert into Fahrenheit, please enter Temperature in Kelvin: 123
Temperature in Fahrenheit: -238.27
--------------------------------
Process exited after 5.047 seconds with return value 0
Press any key to continue . . .
```

DIFFICULTIES/OBSERVATIONS: This program utilizes the float function KelvinToFahrenheit. This function is declared to be defined later in the program by giving the calculations for converting the given temperature from Kelvin to Fahrenheit. This program is also user-input, meaning the user can put any values in the program for conversion.

3. Create a C++ program that has a function that will calculate the distance between two points.

```cpp
#include <iostream>
#include <cmath>

float distancecalc (float X1, float Y1, float X2, float Y2);

int main() {
    float X1, X2, Y1, Y2;
    std::cout << "Distance Calculator\n";
    std::cout << "Enter x value of the first point: ";
    std::cin >> X1;
    std::cout << "Enter y value of the first point: ";
    std::cin >> Y1;
    std::cout << "Enter x value of the second point: ";
    std::cin >> X2;
    std::cout << "Enter y value of the second point: ";
    std::cin >> Y2;
    std::cout << "\nThe distance between the two points is: " << distancecalc (X1, Y1, X2, Y2);
    return 0;
}

float distancecalc (float X1, float Y1, float X2, float Y2){
    float d = std::sqrt(std::pow(X2 - X1, 2.0f) + std::pow(Y2 - Y1, 2.0f));
    return d;
}
```

```
Distance Calculator
Enter x value of the first point: 5
Enter y value of the first point: 10
Enter x value of the second point: 15
Enter y value of the second point: 20

The distance between the two points is: 14.1421
--------------------------------
Process exited after 9.114 seconds with return value 0
Press any key to continue . . .
```

DIFFICULTIES/OBSERVATIONS: I personally struggled with this program since I am heavily unfamiliar with how I am supposed to set things up for point calculations. I got confused along the way and sought some help from the search engine and debugged the code afterwards. After understanding the code, I can now piece together that having X1, Y1, X2, and Y2, the parameters, declared, they are able to store the values that are given by the user. Then, the float function distancecalc will be defined down the line by providing the specific calculations that are required for solving the problem, which is kind of similar from the number 2 activity.

4. Modify the code given in ILO B and add the following functions:
a. A function to compute for the area of a triangle
b. A function to compute for the perimeter of a triangle
c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```cpp
#include <iostream>
#include <cmath>

class Triangle {
private:
    double totalAngle, angleA, angleB, angleC;
    double sideA, sideB, sideC;

public:
    Triangle(double A, double B, double C);

    void setAngles(double A, double B, double C);
    void setSides(double a, double b, double c);

    bool validateTriangle() const;
    double area() const;
    double perimeter() const;
    void triangletype() const;
};

int main() {
    Triangle set1(40, 30, 110);

    set1.setSides(5, 6, 7);

    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle\n";
        std::cout << "Perimeter = " << set1.perimeter() << std::endl;
        std::cout << "Area = " << set1.area() << std::endl;
        set1.triangletype();
    }
    else {
        std::cout << "The shape is NOT a valid triangle\n";
    }

    return 0;
}

Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;

```

```
45      sideA = 0;
46      sideB = 0;
47      sideC = 0;
48  }
49
50  void Triangle::setAngles(double A, double B, double C) {
51      angleA = A;
52      angleB = B;
53      angleC = C;
54      totalAngle = A + B + C;
55  }
56
57  void Triangle::setSides(double a, double b, double c) {
58      sideA = a;
59      sideB = b;
60      sideC = c;
61  }
62
63  bool Triangle::validateTriangle() const {
64      return (angleA > 0 &&
65              angleB > 0 &&
66              angleC > 0 &&
67              totalAngle == 180);
68  }
69
70  double Triangle::perimeter() const {
71      return sideA + sideB + sideC;
72  }
73
74  double Triangle::area() const {
75      double s = perimeter() / 2.0;
76      return std::sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
77  }
78
79  void Triangle::triangletype() const {
80      if (angleA < 90 && angleB < 90 && angleC < 90)
81          std::cout << "This triangle is Acute." << std::endl;
82      else if (angleA > 90 || angleB > 90 || angleC > 90)
83          std::cout << "This triangle is Obtuse." << std::endl;
84      else
85          std::cout << "This triangle is classified as Others." << std::endl;
86  }
```

```
The shape is a valid triangle
Perimeter = 18
Area = 14.6969
This triangle is Obtuse.

--------------------------------
Process exited after 0.2179 seconds with return value 0
Press any key to continue . . .
```

DIFFICULTIES/OBSERVATIONS: I personally always have difficulties when it comes to integrating mathematical operations in programming because they use different kinds of symbols. Because of this, I opted to use the search engine to guide me on how to apply them in C++. I also struggled figuring out how to group the functions together and to figure each of them out separately.

CONCLUSION

- Because of this activity, I gained some understanding of basic C++ programming concepts, which include the use of variables, functions, conditional statements, classes, and math operations. The processes taught me how important it is to be able to organize the code in a way that allows its reusability and the use of programming in solving various problems. The additional supplementary activities enabled me to utilize these concepts to design programs such as the value swap, temperature conversion, the distance calculation program, and the improvement of the triangle program from ILO B with the addition of more functions. Some of the difficulties I faced in doing the work included variable misspellings, struggling to construct the definitions and functions, and logical errors. Despite these, I managed to solve most of the issues, which has enhanced my debugging abilities a little bit. Generally, I feel that I have done pretty well in this exercise and have been able to meet all the expected learning outcomes. Nonetheless, I feel I still need to improve my coding efficiency, debugging skills, and learn more about C++ programming concepts.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **AI is used for grammar correction and code debugging**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **AI is only used whenever there is a need for code debugging. AI is also used for grammar correction and improvement. Aside from the stated uses, all content, thought, and authorship for this work is entirely my own.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.